

GOERTZEL ALGORITHM

```
module goertzel
(
    input wire clk,reset,
    output reg signed [15:0] maxcoeff,
    output wire signed [15:0] COEFF_tb,
    output reg signed [63:0] power_tb,
    output reg signed [63:0] max_power,
    output wire signed [63:0] power

);

reg [6:0] addr = 7'd0;
wire signed [15:0] sample;

localparam integer Fs = 8000;
localparam integer N = 128;

reg signed [15:0] COEFF [0:7];

initial
begin
    COEFF[0] = 16'sd28106; // 697 Hz
    COEFF[1] = 16'sd27018; // 770 Hz
    COEFF[2] = 16'sd25330; // 852 Hz
    COEFF[3] = 16'sd24126; // 941 Hz
    // COEFF[4] = 16'sd19478; // 1209 Hz
    // COEFF[5] = 16'sd16846; // 1336 Hz
    // COEFF[6] = 16'sd12490; // 1477 Hz
    // COEFF[7] = 16'sd9220; // 1633 Hz
end

reg [1:0] i = 2'd0;
reg signed [31:0] s1 = 0;
reg signed [31:0] s2 = 0;

wire signed [31:0] s_next;
wire signed [47:0] s1_prod;
assign s1_prod = COEFF[i] * s1;
reg [6:0] count = 0;

assign COEFF_tb=COEFF[i];
```

```
assign s_next = {{16{sample[15]}}}, sample} + (s1_prod >>> 14) - s2;
```

```
assign power =  
    (s_next * s_next)  
    + (s1 * s1)  
    - (((COEFF[i] * s_next * s1) >>> 14));
```

```
always @(posedge clk)  
begin
```

```
    if(reset)  
    begin  
        addr    <= 7'd0;  
        count    <= 7'd0;  
        i        <= 2'd0;  
        s1        <= 32'sd0;  
        s2        <= 32'sd0;  
        max_power <= 64'sd0;  
        maxcoeff  <= 16'sd0;  
        power_tb  <= 64'sd0;
```

```
    end  
    else  
    begin
```

```
        addr <= addr + 1'b1;
```

```
        if(count == 7'd127)  
        begin  
            power_tb <= power;
```

```
            if(power > max_power)  
            begin  
                max_power <= power;  
                maxcoeff <= COEFF[i];
```

```
        end
```

```
        count <= 7'd0;  
        s1    <= 32'sd0;  
        s2    <= 32'sd0;  
        addr  <= 7'd0;
```

```
        if (i == 2'd3)  
        begin  
            i <= 2'd0;  
        end  
        else  
        begin  
            i <= i + 1'b1;
```

```

        end
    end
    else
    begin

        s2 <= s1;
        s1 <= s_next;

        count <= count + 1'b1;

    end
end
end

sinwave sw(
    .addr(addr),
    .sample(sample)
);

endmodule

module goertzel1
(

    input wire clk,reset,
    output reg signed [15:0] maxcoeff,
    output wire signed [15:0] COEFF_tb,
    output reg signed [63:0] power_tb,
    output reg signed [63:0] max_power,
    output wire signed [63:0] power

);

    reg [6:0] addr = 7'd0;
    wire signed [15:0] sample;

    localparam integer Fs = 8000;
    localparam integer N = 128;

    reg signed [15:0] COEFF [0:7];

    initial
    begin
        // COEFF[0] = 16'sd28106; // 697 Hz
        // COEFF[1] = 16'sd27018; // 770 Hz
        // COEFF[2] = 16'sd25330; // 852 Hz
        // COEFF[3] = 16'sd24126; // 941 Hz
    end

```

```

COEFF[0] = 16'sd19478; // 1209 Hz
COEFF[1] = 16'sd16846; // 1336 Hz
COEFF[2] = 16'sd12490; // 1477 Hz
COEFF[3] = 16'sd9220; // 1633 Hz
end

```

```

reg [1:0] i = 2'd0;
reg signed [31:0] s1 = 0;
reg signed [31:0] s2 = 0;

```

```

wire signed [31:0] s_next;
wire signed [47:0] s1_prod;
assign s1_prod = COEFF[i] * s1;
reg [6:0] count = 0;

```

```

assign COEFF_tb=COEFF[i];
assign s_next = {{16{sample[15]}}}, sample} + (s1_prod >>> 14) - s2;

```

```

assign power =
    (s_next * s_next)
    + (s1 * s1)
    - (((COEFF[i] * s_next * s1) >>> 14));

```

```

always @(posedge clk)
begin

```

```

    if(reset)
    begin
        addr    <= 7'd0;
        count   <= 7'd0;
        i       <= 2'd0;
        s1      <= 32'sd0;
        s2      <= 32'sd0;
        max_power <= 64'sd0;
        maxcoeff <= 16'sd0;
        power_tb <= 64'sd0;

```

```

    end
    else
    begin

```

```

        addr <= addr + 1'b1;

```

```

        if(count == 7'd127)
        begin
            power_tb<=power;

```

```

            if(power>max_power)
            begin

```

```
    max_power<=power;
    maxcoeff<=COEFF[i];
```

```
end
```

```
count <= 7'd0;
s1    <= 32'sd0;
s2    <= 32'sd0;
addr  <= 7'd0;
```

```
if (i == 2'd3)
begin
    i <= 2'd0;
end
else
begin
    i <= i + 1'b1;
end
end
else
begin
```

```
    s2 <= s1;
    s1 <= s_next;
```

```
    count <= count + 1'b1;
```

```
end
end
end
```

```
sinwave sw(
    .addr(addr),
    .sample(sample)
);
```

```
endmodule
```

TESTBENCH

```
`timescale 1ns/1ps
```

```
module tb_goertzel;
```

```
reg clk;
reg reset;
```

```
reg [7:0] key [0:3][0:3];
```

```

integer j,k;

initial begin
    key[0][0] = "1"; key[0][1] = "2"; key[0][2] = "3"; key[0][3] = "A";
    key[1][0] = "4"; key[1][1] = "5"; key[1][2] = "6"; key[1][3] = "B";
    key[2][0] = "7"; key[2][1] = "8"; key[2][2] = "9"; key[2][3] = "C";
    key[3][0] = "*"; key[3][1] = "0"; key[3][2] = "#"; key[3][3] = "D";
end

wire signed [15:0] maxcoeff;
    wire signed [63:0] power ;
    wire signed [63:0] power_tb ;
    wire signed [63:0] max_power ;
    wire signed [15:0] COEFF_tb;

reg reset1;

wire signed [15:0] maxcoeff1;
    wire signed [63:0] power1 ;
    wire signed [63:0] power_tb1 ;
    wire signed [63:0] max_power1 ;
    wire signed [15:0] COEFF_tb1;

goertzel dut(
.clk(clk),
.reset(reset),
.maxcoeff(maxcoeff),
.power(power),
.power_tb(power_tb),
.COEFF_tb(COEFF_tb),
.max_power(max_power)

);

goertzel1 dut1(
.clk(clk),
.reset(reset1),
.maxcoeff(maxcoeff1),
.power(power1),
.power_tb(power_tb1),
.COEFF_tb(COEFF_tb1),
.max_power(max_power1)

);

always #41.667 clk = ~clk;

integer F;

```

```

initial begin
    $dumpfile("wave.vcd");
    $dumpvars(0, tb_goertzel);
    clk = 0;
    reset=1;
    #100
    reset=0;
    $monitor("time=%0t , i=%0d , coeff=%0d , power=%0d , peakpower=%0d",
        $time, dut.i, COEFF_tb, power_tb, max_power);
    #50000;
case(maxcoeff)
    16'sd28106 :begin
        F = 697;
        j=0;
    end
    16'sd27018 : begin
        F = 770;
        j=1;
    end
    16'sd25330 : begin
        F = 852;
        j=2;
    end
    16'sd24126 :
    begin
        F = 941;
        j=3;
    end
    // 16'sd19478 : F = 1209;
    // 16'sd16846 : F = 1336;
    // 16'sd12490 : F = 1477;
    // 16'sd9220 : F = 1633;
    default : F = 0;
endcase

```

```

    $display("det freq: %0d Hz", F);
    $display("power peak: %0d", max_power);

```

```

    clk = 0;
    reset1=1;
    #100
    reset1=0;
    $monitor("time=%0t , i=%0d , coeff=%0d , power=%0d , peakpower=%0d",
        $time, dut1.i, COEFF_tb1, power_tb1, max_power1);
    #50000;
case(maxcoeff1)
    // 16'sd28106 : F = 697;
    // 16'sd27018 : F = 770;
    // 16'sd25330 : F = 852;

```

```

// 16'sd24126 : F = 941;
16'sd19478 : begin
    F = 1209;
    k=0;
end

16'sd16846 : begin
    F = 1336;
    k=1;
end
16'sd12490 :begin
    F = 1477;
    k=2;
end
16'sd9220 :begin

    F = 1633;
    k=3;
end
    default    : F = 0;
endcase

    $display("det freq: %0d Hz", F);
    $display("power peak: %0d", max_power1);

$display("key is: %0s", key[j][k]);


    $finish;

end

```

```

endmodule

```

```

SINNWAVE.v

```

```

module sinwave (
    input  wire [6:0] addr,
    output reg signed [15:0] sample
);
    always @(*) begin
        case (addr)
7'd0: sample = 16'sd0;
7'd1: sample = 16'sd24234;

```


7'd2: sample = 16'sd23492;
7'd3: sample = 16'sd5711;
7'd4: sample = -16'sd1629;
7'd5: sample = 16'sd8565;
7'd6: sample = 16'sd13843;
7'd7: sample = -16'sd3353;
7'd8: sample = -16'sd27635;
7'd9: sample = -16'sd29997;
7'd10: sample = -16'sd7655;
7'd11: sample = 16'sd12140;
7'd12: sample = 16'sd9732;
7'd13: sample = -16'sd1344;
7'd14: sample = 16'sd3318;
7'd15: sample = 16'sd21558;
7'd16: sample = 16'sd26423;
7'd17: sample = 16'sd4996;
7'd18: sample = -16'sd21374;
7'd19: sample = -16'sd24908;
7'd20: sample = -16'sd8248;
7'd21: sample = 16'sd1576;
7'd22: sample = -16'sd7232;
7'd23: sample = -16'sd15011;
7'd24: sample = -16'sd848;
7'd25: sample = 16'sd24629;
7'd26: sample = 16'sd31642;
7'd27: sample = 16'sd11939;
7'd28: sample = -16'sd9921;
7'd29: sample = -16'sd10654;
7'd30: sample = 16'sd756;
7'd31: sample = -16'sd1148;
7'd32: sample = -16'sd19190;
7'd33: sample = -16'sd27896;
7'd34: sample = -16'sd9857;
7'd35: sample = 16'sd17907;
7'd36: sample = 16'sd25735;
7'd37: sample = 16'sd10840;
7'd38: sample = -16'sd1164;
7'd39: sample = 16'sd5819;
7'd40: sample = 16'sd15643;
7'd41: sample = 16'sd4831;
7'd42: sample = -16'sd21068;
7'd43: sample = -16'sd32520;
7'd44: sample = -16'sd16080;
7'd45: sample = 16'sd7225;
7'd46: sample = 16'sd11260;
7'd47: sample = 16'sd85;
7'd48: sample = -16'sd733;
7'd49: sample = 16'sd16484;
7'd50: sample = 16'sd28634;
7'd51: sample = 16'sd14452;
7'd52: sample = -16'sd13916;
7'd53: sample = -16'sd25919;

7'd54: sample = -16'sd13391;
7'd55: sample = 16'sd391;
7'd56: sample = -16'sd4407;
7'd57: sample = -16'sd15758;
7'd58: sample = -16'sd8494;
7'd59: sample = 16'sd17058;
7'd60: sample = 16'sd32601;
7'd61: sample = 16'sd19954;
7'd62: sample = -16'sd4110;
7'd63: sample = -16'sd11487;
7'd64: sample = -16'sd1117;
7'd65: sample = 16'sd2275;
7'd66: sample = -16'sd13541;
7'd67: sample = -16'sd28637;
7'd68: sample = -16'sd18661;
7'd69: sample = 16'sd9502;
7'd70: sample = 16'sd25425;
7'd71: sample = 16'sd15804;
7'd72: sample = 16'sd729;
7'd73: sample = 16'sd3079;
7'd74: sample = 16'sd15393;
7'd75: sample = 16'sd11745;
7'd76: sample = -16'sd12715;
7'd77: sample = -16'sd31878;
7'd78: sample = -16'sd23445;
7'd79: sample = 16'sd651;
7'd80: sample = 16'sd11287;
7'd81: sample = 16'sd2269;
7'd82: sample = -16'sd3441;
7'd83: sample = 16'sd10468;
7'd84: sample = 16'sd27928;
7'd85: sample = 16'sd22372;
7'd86: sample = -16'sd4779;
7'd87: sample = -16'sd24239;
7'd88: sample = -16'sd17980;
7'd89: sample = -16'sd2166;
7'd90: sample = -16'sd1909;
7'd91: sample = -16'sd14599;
7'd92: sample = -16'sd14509;
7'd93: sample = 16'sd8164;
7'd94: sample = 16'sd30369;
7'd95: sample = 16'sd26443;
7'd96: sample = 16'sd3063;
7'd97: sample = -16'sd10625;
7'd98: sample = -16'sd3462;
7'd99: sample = 16'sd4214;
7'd100: sample = -16'sd7370;
7'd101: sample = -16'sd26552;
7'd102: sample = -16'sd25489;
7'd103: sample = -16'sd127;
7'd104: sample = 16'sd22367;
7'd105: sample = 16'sd19823;

```

7'd106: sample = 16'sd3872;
7'd107: sample = 16'sd962;
7'd108: sample = 16'sd13442;
7'd109: sample = 16'sd16730;
7'd110: sample = -16'sd3534;
7'd111: sample = -16'sd28114;
7'd112: sample = -16'sd28853;
7'd113: sample = -16'sd6930;
7'd114: sample = 16'sd9485;
7'd115: sample = 16'sd4616;
7'd116: sample = -16'sd4595;
7'd117: sample = 16'sd4349;
7'd118: sample = 16'sd24572;
7'd119: sample = 16'sd27937;
7'd120: sample = 16'sd5085;
7'd121: sample = -16'sd19839;
7'd122: sample = -16'sd21247;
7'd123: sample = -16'sd5785;
7'd124: sample = -16'sd291;
7'd125: sample = -16'sd11999;
7'd126: sample = -16'sd18371;
7'd127: sample = -16'sd1046;

```

```

        default: sample = 16'sd0;
    endcase
end
endmodule

```

```

SINWAVE.py
import math

```

```

FS = 8000.0
FREQ1 = 697.0
FREQ2 = 1633.0
AMP = 16383

```

```

for n in range(128):
    v = round(
        AMP * math.sin(2 * 3.14 * FREQ1 * n / FS) +
        AMP * math.sin(2 * 3.14 * FREQ2 * n / FS)
    )

    if v >= 0:
        print(f"7'd{n}: sample = 16'sd{v};")
    else:
        print(f"7'd{n}: sample = -16'sd{abs(v)};")

```

OUTPUT

```
kevin@kevin-ThinkPad-T450:~/Desktop/icePie-examples/dtmf/verilog/decoder$ vvp sim
VCD info: dumpfile wave.vcd opened for output.
time=100000 , i=0 , coeff=28106 , power=0 , peakpower=0
time=10708000 , i=1 , coeff=27018 , power=1012822134133 , peakpower=1012822134133
time=21375000 , i=2 , coeff=25330 , power=4055290577 , peakpower=1012822134133
time=32042000 , i=3 , coeff=24126 , power=2902790070 , peakpower=1012822134133
time=42709000 , i=0 , coeff=28106 , power=467376338 , peakpower=1012822134133
det freq: 697 Hz
power peak: 1012822134133
time=50200000 , i=0 , coeff=19478 , power=0 , peakpower=0
time=60792000 , i=1 , coeff=16846 , power=195249830 , peakpower=195249830
time=71459000 , i=2 , coeff=12490 , power=81353615 , peakpower=195249830
time=82126000 , i=3 , coeff=9220 , power=840986776 , peakpower=840986776
time=92792000 , i=0 , coeff=19478 , power=1058816820960 , peakpower=1058816820960
det freq: 1633 Hz
power peak: 1058816820960
key is: A
```